

Unified Modeling Language (UML)

Object Oriented Modeling and Programming

What is UML?

If UML is just boxes and arrows...
why do we need tools?

Learning Objectives

By the end of this session, you will be able to:

- **Explain** the role of UML in Object-Oriented Analysis and Design (OOAD).
- **Identify and differentiate** between UML structural and behavioral diagrams.
- **Model system structure** using Class, Object, Component, and Deployment diagrams.
- **Analyze system behavior** using Use Case, Activity, and State Machine diagrams.
- **Design object interactions** using Sequence, Communication, and Timing diagrams.
- **Apply UML tools** (e.g., StarUML, Visual Paradigm, Draw.io) to create clear and consistent software models.

What is UML

- **UML (Unified Modeling Language)** is a standard visual language used to model, design, and document object-oriented systems.
- UML is not a programming language, but a visual language.
- Introduced by, Booch, Rumbaugh and Jacobson (2004,2005).
- The UML has 13 diagram types.
- Supports the creation of many different types of system models.

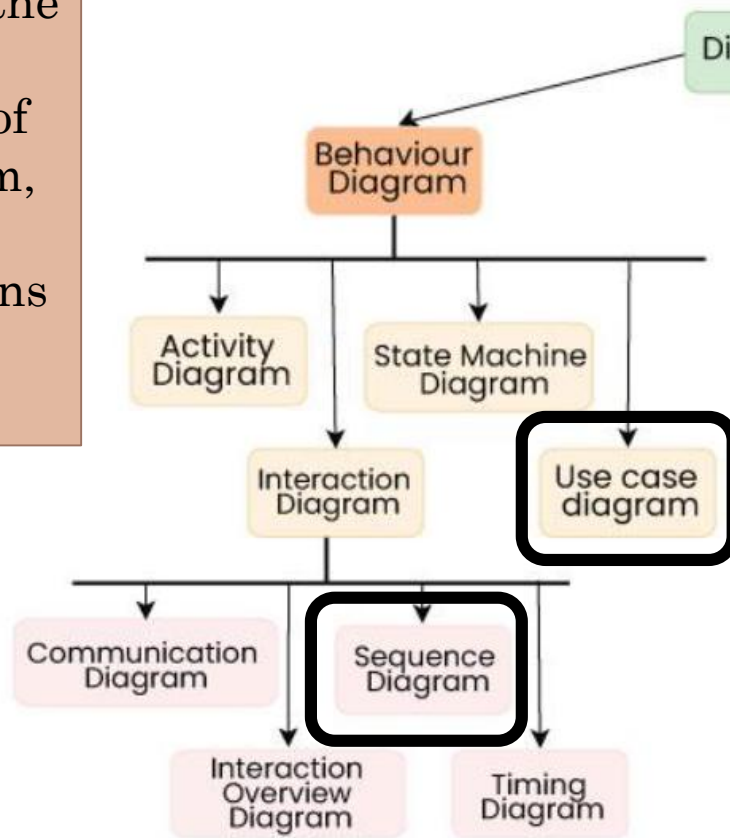


Why UML?

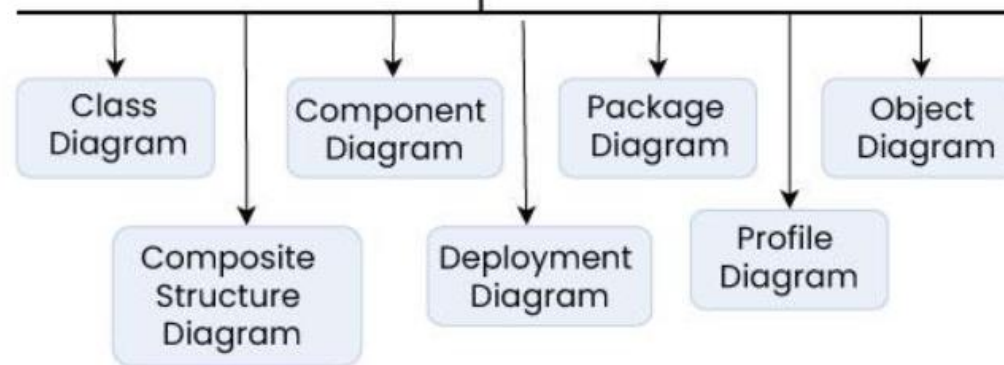
- Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate amongst them.
- Businessmen do not understand code. So UML becomes essential to communicate with non-programmers about essential requirements, functionalities, and processes.
- Time is saved: Teams can visualize processes, user interactions, and the static structure of the system.

Types of UML Diagrams

Describe the dynamic behavior of the system, including interactions between elements.



Visual representations that depict the static aspects of a system, including its classes, objects, components, and their relationships



Tools for creating UML Diagrams

- **Lucidchart:** A web-based diagramming tool, user-friendly and collaborative
- **Draw.io:** A free, web-based diagramming tool
- **Visual Paradigm:** Offers both online and desktop versions and supports a wide range of UML diagrams (supports forward and reverse engineering).
- **StarUML:** An open-source UML modeling tool with a user-friendly interface. It supports the standard UML 2.x diagrams and allows users to customize and extend its functionality through plugins.
- **ArgoUML, PlantUML, Rational Rose...etc**

What the System is Supposed to do?

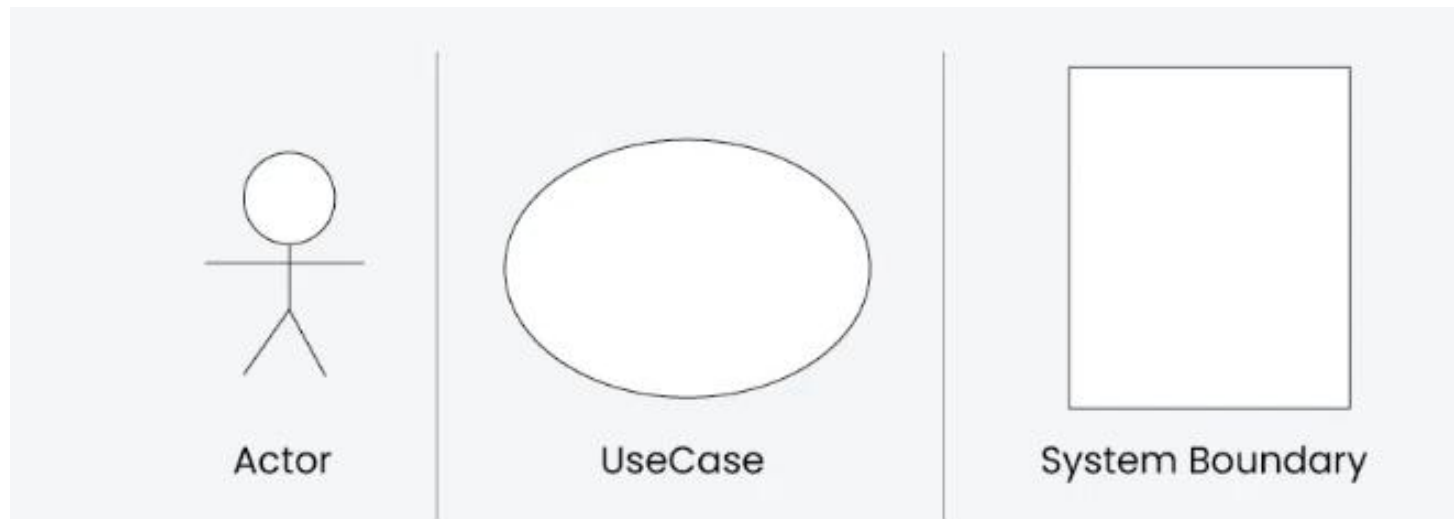
Before designing classes, what must we understand first?

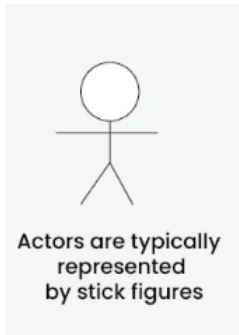
Use-case Diagrams

Defines what the system should do (user perspective)... Not *how* it does it

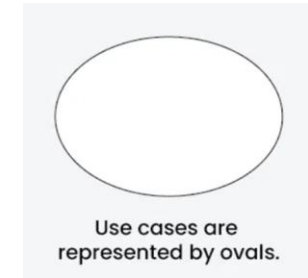
Use-Case Diagrams

Represents the interaction between actors (users or external systems) and a system under consideration to accomplish specific goals.





Use Case Diagram Notations



Actors

- External entities that interact with the system
- Named by noun
- Plays a role in the business
- Include users, other systems, or hardware devices
- Initiate use cases

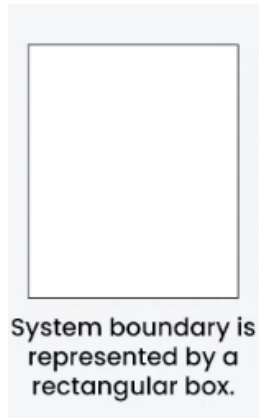
Use Cases

- Represent specific things the system can do
- System function (process - automated or manual)
- Named by Verb + Noun (or Noun Phrase)
- Do something
- Each Actor must be linked to a use case, while some use cases may not be linked to actors

Use Case Diagram Notations

System Boundary

- A visual representation of the scope or limits of the system
- Defines what is inside the system and what is outside.
- Establish a clear distinction between the elements that are part of the system and those that are external to it.



Relationships

- Represent specific things the system can do.
- Depicts the interactions between actors and use cases.

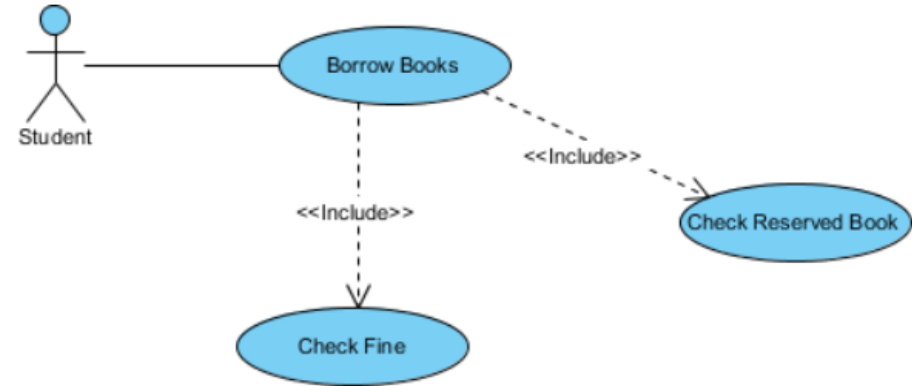
Relationships

Use Case Diagrams

Relationships

Association Relationship

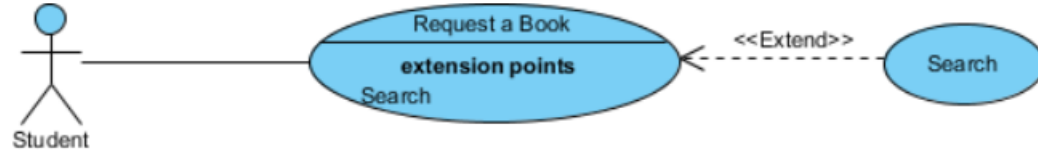
- Represents a communication or interaction between an actor and a use case.
- Depicted by a line connecting the actor to the use case.
- Signifies that the actor is involved in the functionality described by the use case.



Include Relationship

- Indicates that a use case includes the functionality of another use case.
- Denoted by a dashed arrow pointing from the including use case to the included use case.
- The stereotype "<<include>>" identifies the relationship as an include relationship.

Relationships

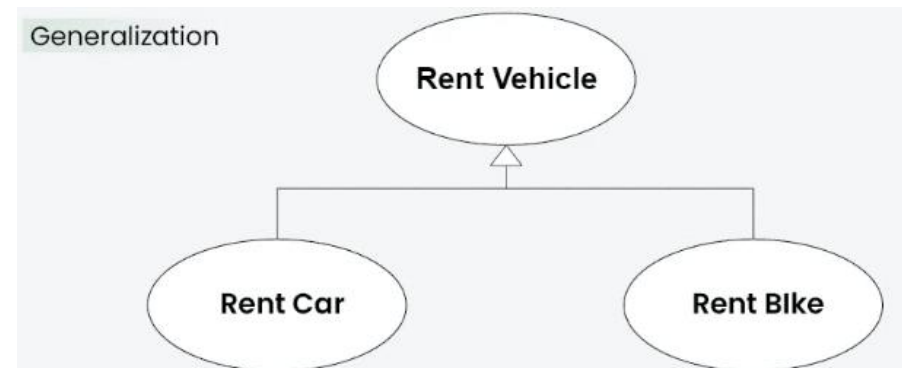


Extend Relationship

- Illustrates a use case that can be extended by another use case under specific conditions.
- Represented by a dashed arrow with the keyword "extend."
- Useful for handling optional or exceptional behavior.
- The stereotype "<<extends>>"

Generalization Relationship

- Establishes an "is-a" connection between two use cases, indicating that one use case is a specialized version of another.
- Represented by an arrow pointing from the specialized use case to the general use case.



How to Draw?

- **Step 1: Identify Actors:** Determine who or what interacts with the system (users, other systems, or external entities)
- **Step 2: Identify Use Cases:** Identify the main functionalities or actions the system must perform. Each use case should represent a specific piece of functionality.
- **Step 3: Connect Actors and Use Cases:** Draw lines (associations) between actors and the use cases they are involved in. This represents the interactions between actors and the system.
- **Step 4: Add System Boundary:** Draw a box around the actors and use cases to represent the system boundary. This defines the scope of the system.
- **Step 5: Define Relationships:** If certain use cases are related or if one use case is an extension of another, indicate the relationships with appropriate notations.
- **Step 6: Review and Refine:** Ensure that it accurately represents the interactions and relationships. Refine as needed.
- **Step 7: Validate:** Share the use case diagram with stakeholders and gather feedback.

Activity

Scenario: Academic and Assessment Management System

How to Identify Actor (Schneider and Winters - 1998)

- Who uses the system?
- Who installs the system?
- Who starts up the system?
- Who maintains the system?
- Who shuts down the system?
- What other systems use this system?
- Who gets information from this system?
- Who provides information to the system?
- Does anything happen automatically at a present time?

How to Identify Use Cases?

- What functions will the actor want from the system?
- Does the system store information? What actors will create, read, update or delete this information?
- Does the system need to notify an actor about changes in the internal state?
- Are there any external events the system must know about? What actor informs the system of those events?

Once we know *what*
the system does, how
do we show *how objects*
interact over time?

Interaction models

Sequence Diagrams

Collaboration Diagrams

How the different parts of a system (objects) communicate with each other in order to perform a task. It focuses on the order of messages over time, not just the static structure.

Activity – Identify the Interactions

When a customer enters a restaurant, they are welcomed and seated by a waiter. The customer then places an order for food and drinks. The waiter records the order and sends the food request to the kitchen and the drink request to the beverage counter.

In the kitchen, the chef prepares the food items, while at the beverage counter, the barista prepares the drinks. Once both the food and drinks are ready, the waiter collects them and serves them to the customer.

After finishing the meal, the customer asks for the bill. The waiter generates the bill using the billing system and presents it to the customer. The customer makes the payment either by cash or card. The waiter processes the payment through the billing system and confirms the transaction.

Interaction models

- All systems involve interaction of some kind.
- Types of interactions:
 - User interactions, involve user inputs and outputs
 - Interactions between the software and other systems in the environment
 - Interactions between the components of a software system
- Show how a set of **actors and objects communicate with each other** to perform the steps of a use case, or of some other piece of functionality.

Interaction model

- Elements in an interaction diagram:
 - **Instances of classes or actors:** instances of classes (i.e. objects) are **shown as boxes** with the class and object identifier underlined
 - **Messages-Different types of communication:** These are **shown as arrows from actor to object, or from object to object.**
- The main objective
 - **To better understand the sequence of messages.**

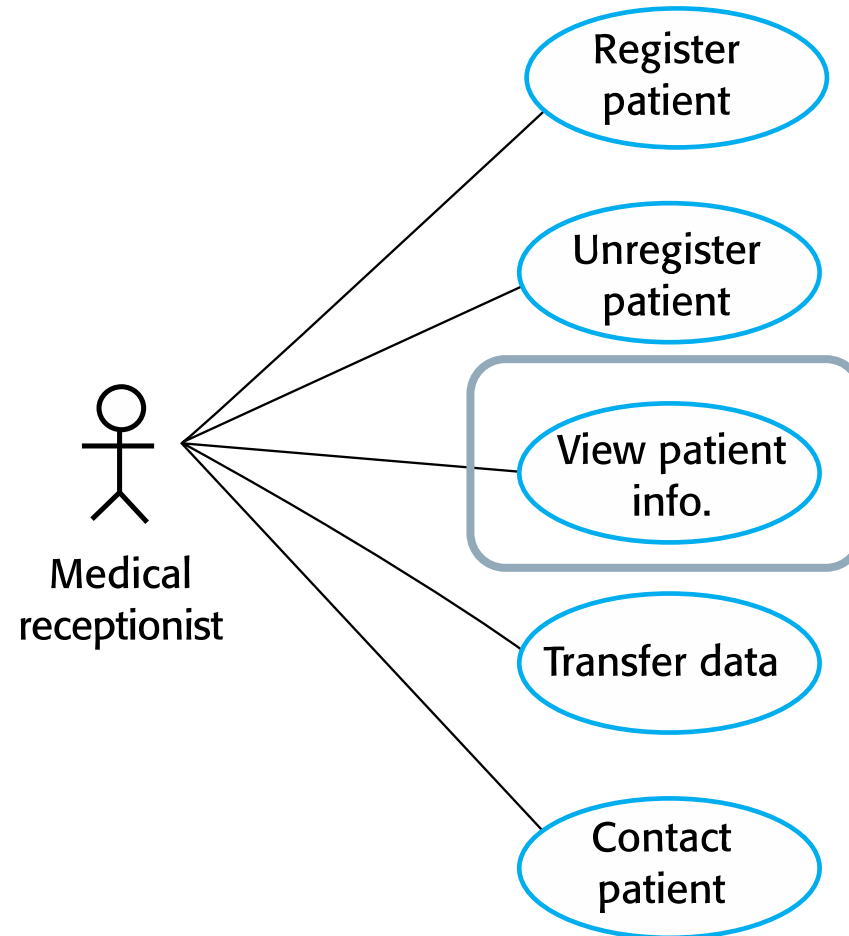
Sequence Diagrams

The interaction that takes place in a collaboration

Sequence Diagrams

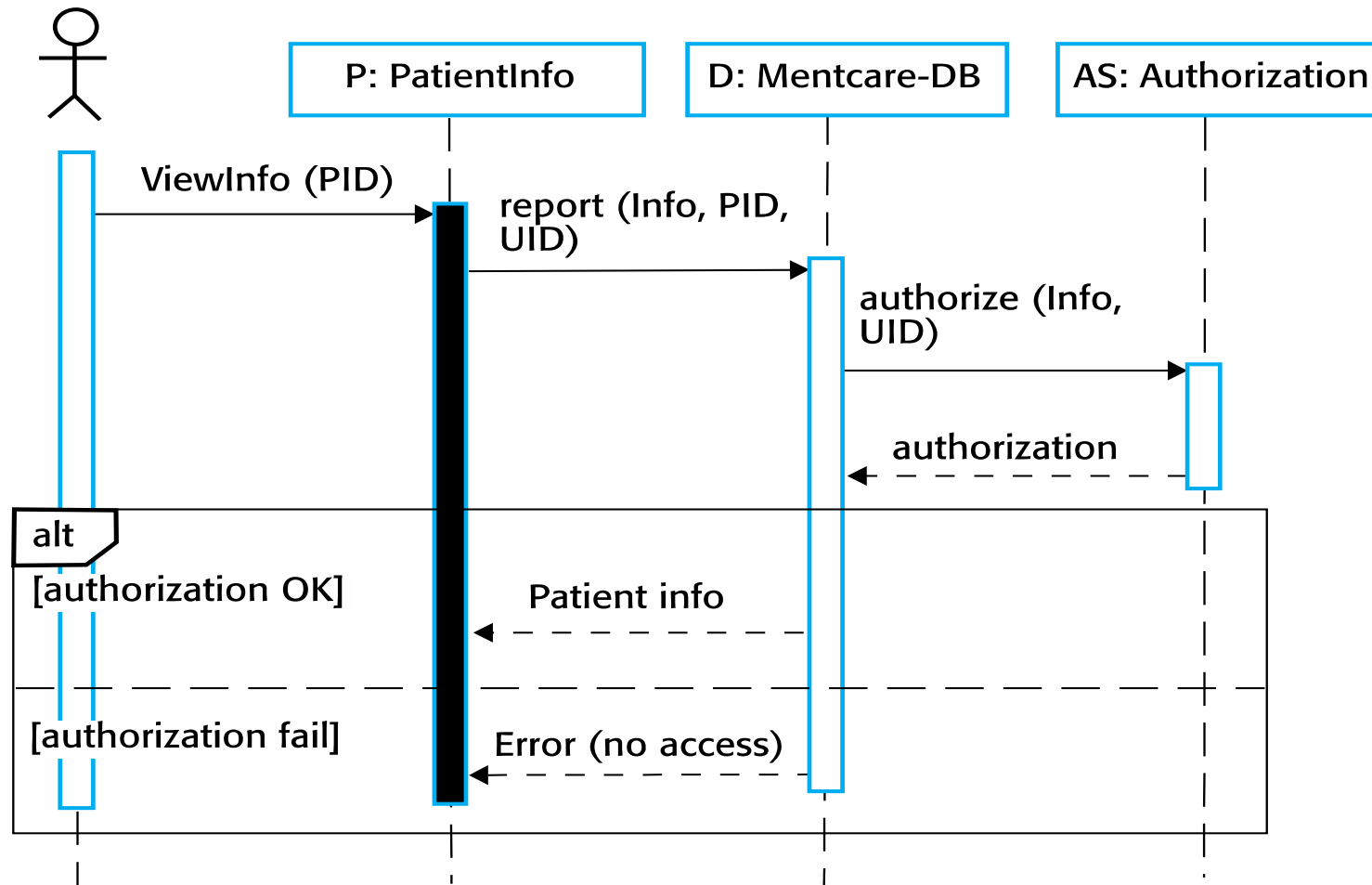
- Commonly used by developers.
- Model the interactions between objects in a single use case.
- Illustrate how the different parts of a system interact with each other to carry out a function, and the order in which the interactions occur when a particular use case is executed.
- Shows different parts of a system work in a 'sequence' to get something done.
- Shows elements as they interact over time and they are organized according to object (horizontally) and time (vertically).
- Object Dimension: The elements that are involved in the interaction
- Time Dimension: Ordering, not duration

Use case: 'Medical Receptionist' (Medical Health System)



Sequence diagram: 'View patient infor'

Medical Receptionist



Notations

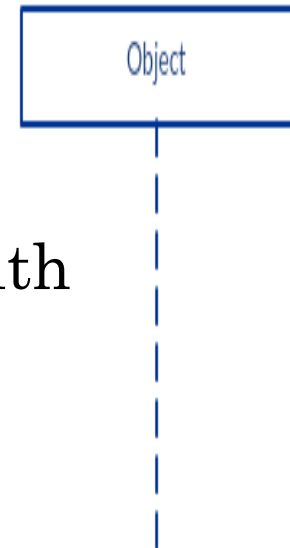
Sequence Diagram

Ref: <https://createely.com/blog/diagrams/sequence-diagram-tutorial/>

Basic Symbols and Components

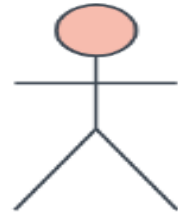
- Represents an individual participant in the Interaction.
- Made up of several of these lifeline notations
- Should be arranged horizontally across the top of the diagram.
- No two lifeline notations should overlap each other.
- Represent different objects or parts that interact with each other in the system during the sequence.

Lifeline Notation



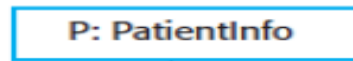
Basic Symbols and Components

- A lifeline notation with an actor element symbol
 - Used when the particular sequence diagram is owned by a use case



Actor
symbol

Shows entities
that interact with
or are external to
the system.

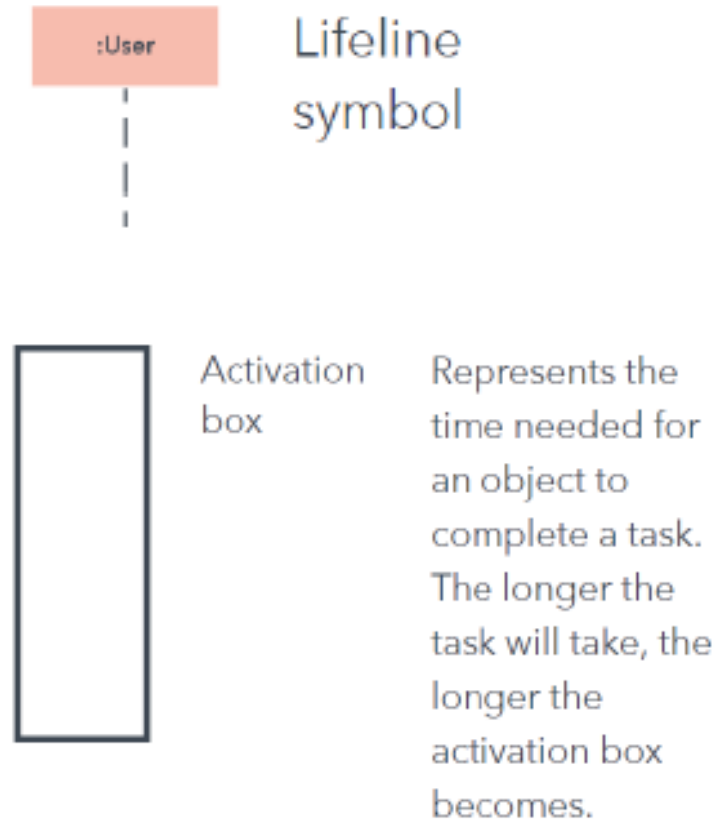


Object
symbol

Represents a
class or object in
UML. The object
symbol
demonstrates
how an object will
behave in the
context of the
system.



Basic Symbols and Components



- Lifelines begin with a labeled rectangle shape or an actor symbol.
- Represents the passage of time as it extends downward.
- This dashed vertical line shows the sequential events that occur to an object during the charted process.

Basic Symbols and Components

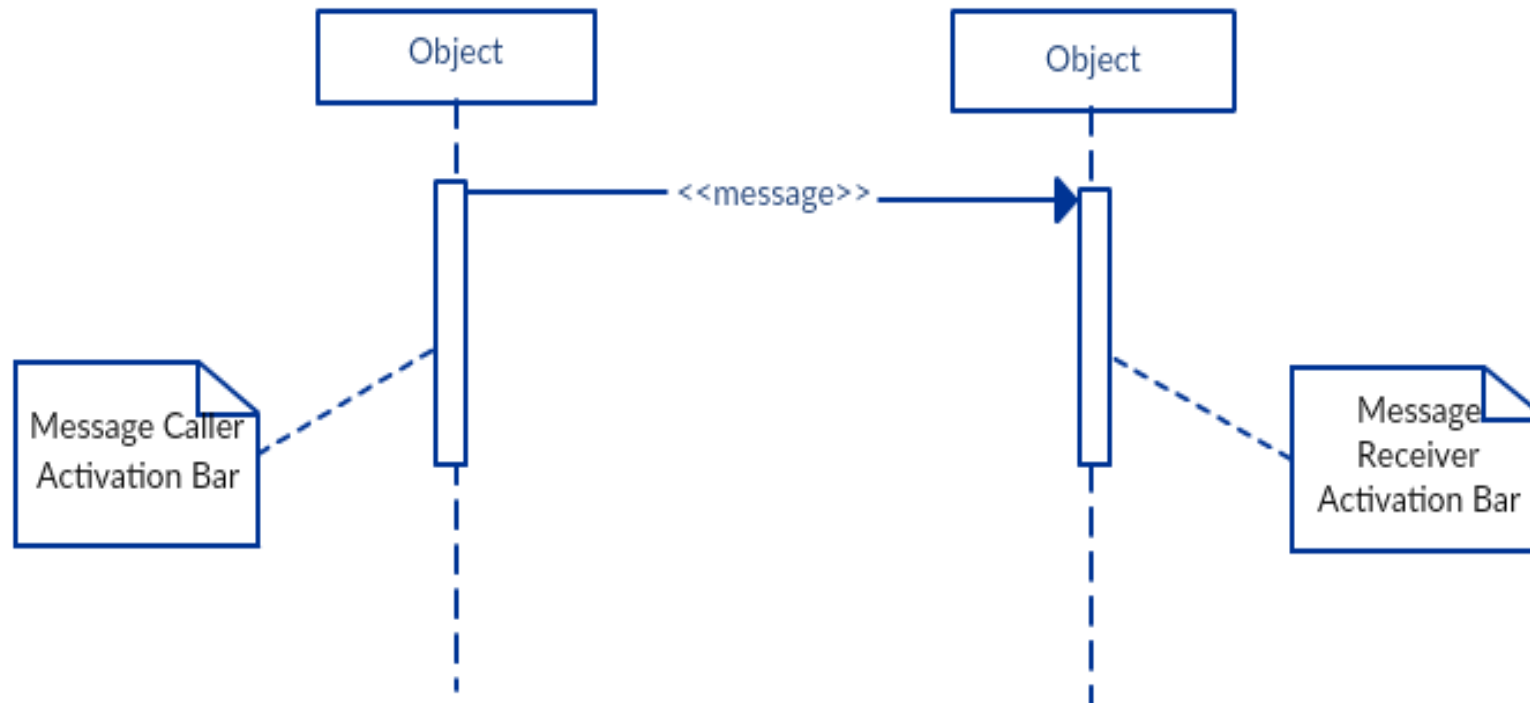
Activation Bars

- A box placed on the lifeline.
- Used to indicate that an object is active (or instantiated) during an interaction between two objects.
- The length of the rectangle indicates the duration of the objects staying active.

Interaction between two objects

- Occurs when one object sends a message to another.
- The use of the activation bar on the lifelines of the
 - Message Caller (the object that sends the message) and
 - Message Receiver (the object that receives the message)
 - Indicates that both are active/is instantiated during the exchange of the message

Activation bar example



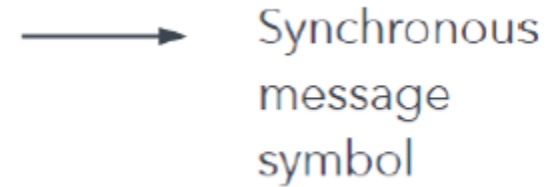
Basic Symbols and Components

Message Arrows

- An arrow
- From the Message Caller to the Message Receiver.
- A message can flow in any direction;
 - from left to right,
 - right to left or
 - back to the Message Caller itself.
- Different arrowheads
 - can indicate the type of message being sent or received.
- A message signature: message description
- *message_name (arguments): return_type*

Common message symbols

- Represented by a **solid line with a solid arrowhead**.
- Sender waits for the receiver to process the message and return before carrying on with another message
- The diagram should show both the call and the reply.



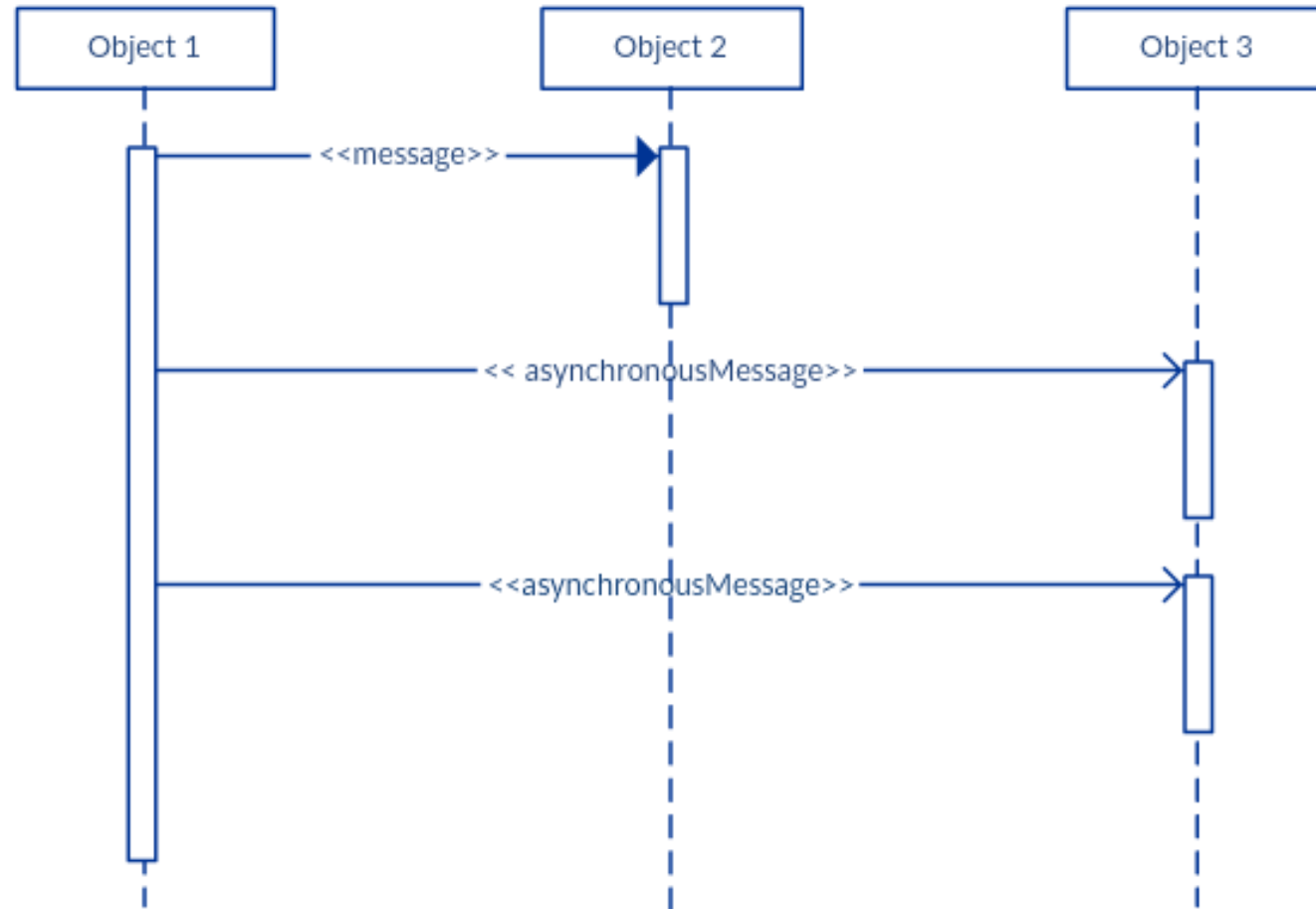
Common message symbols

- Represented by a **solid line with a lined arrowhead.**
- Asynchronous messages **don't require a response before the sender continues.**
- Only the call should be included in the diagram



Asynchronous
message
symbol

Example

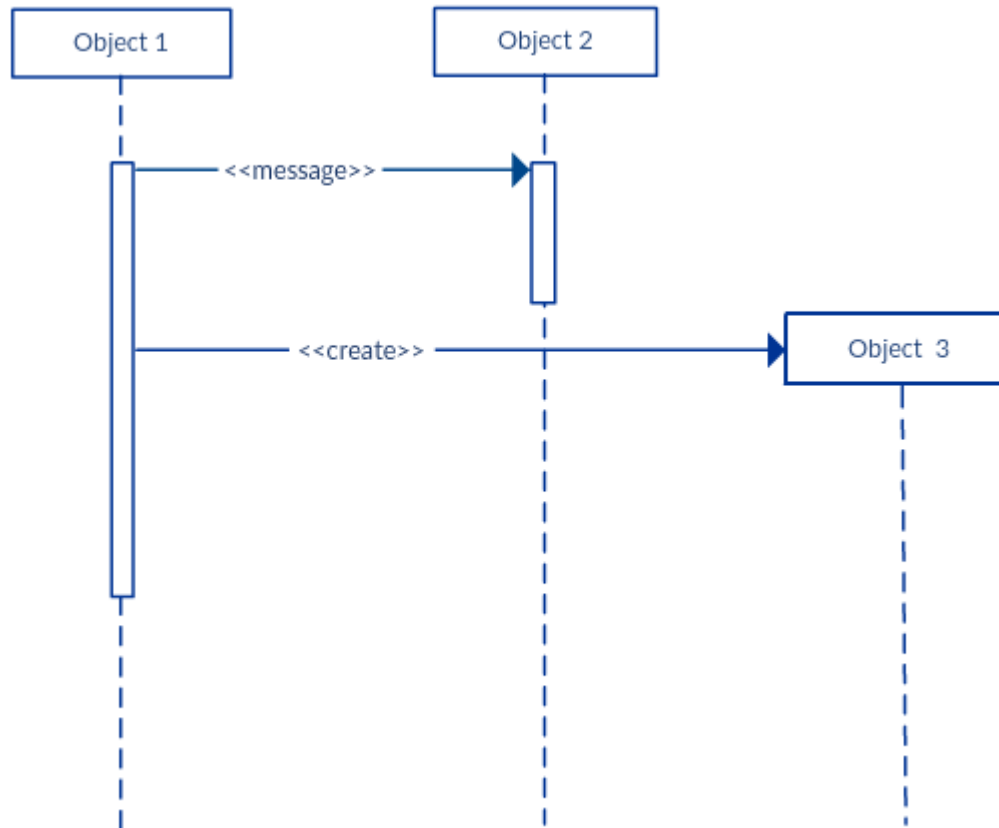


Common message symbols

← - - - Reply
message
symbol

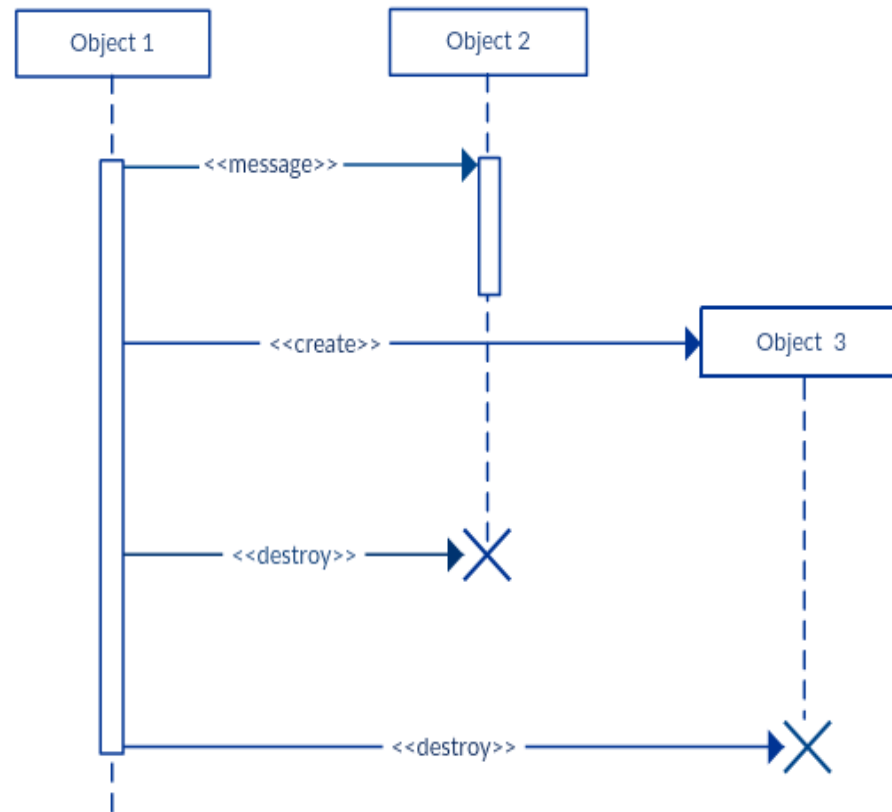
- Represented by a dashed line with a lined arrowhead.
- Indicate that the message receiver is done processing the message and is returning control over to the message caller

Participant Creation Message



- Objects do not necessarily live for the entire duration of the sequence of events.
- Objects or participants can be created according to the message that is being sent.

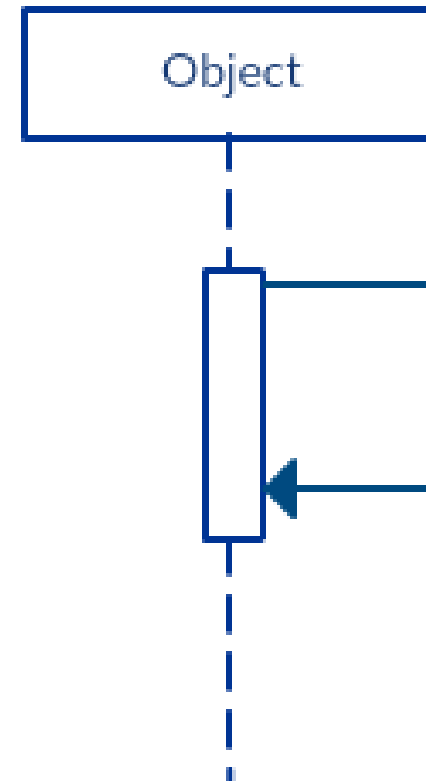
Participant destruction message



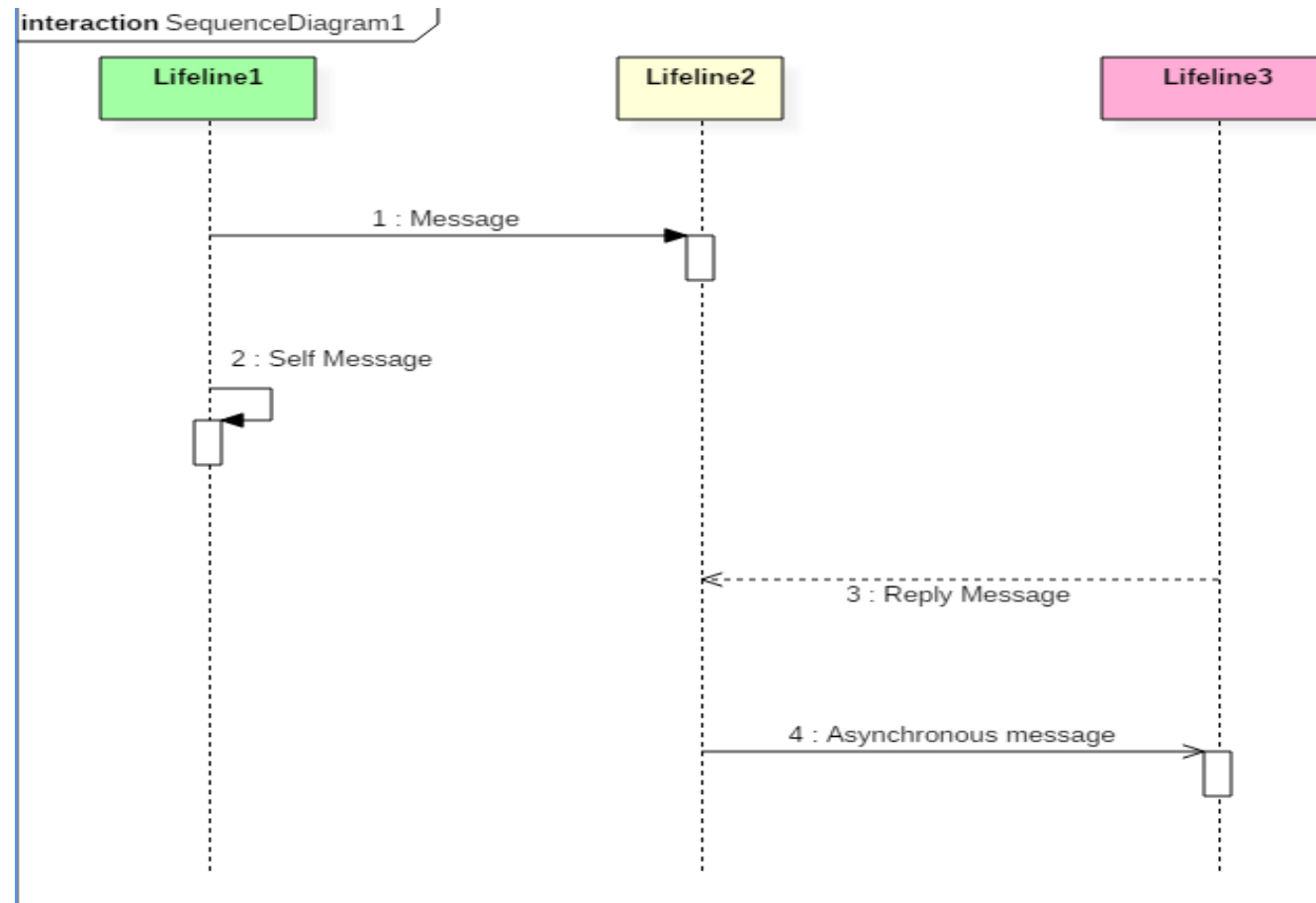
- When no longer needed participants can also be deleted from a sequence diagram.
- This is done by adding an 'X' at the end of the lifeline of the said participant.

Reflexive message

- When an object sends a message to itself, it is called a reflexive message.
- Indicated with a message arrow that starts and ends at the same lifeline.

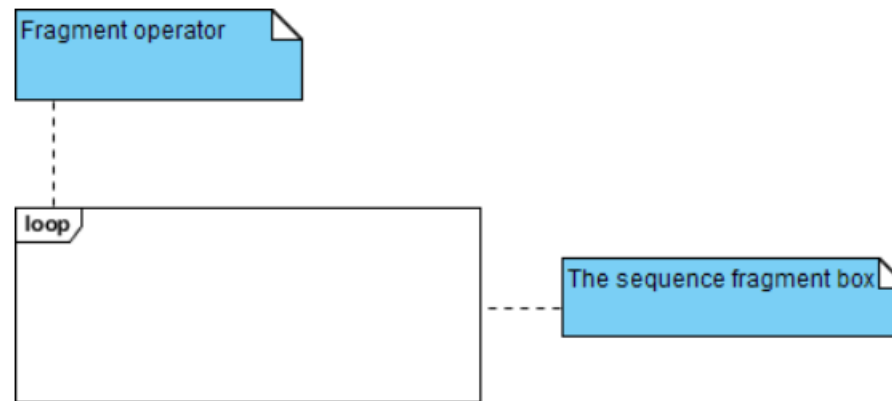


Sequence Modeling



Sequence Fragments

- UML 2.0 introduces sequence (or interaction) fragments.
- Sequence fragments make it easier to create and maintain accurate sequence diagrams
- A sequence fragment is represented as a box, called a combined fragment, which encloses a portion of the interactions within a sequence diagram
- The fragment operator (in the top left corner) indicates the type of fragment
- Fragment types: ref, assert, loop, break, alt, opt, neg

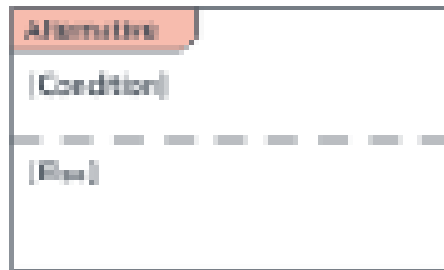


Sequence Fragments

Operator	Fragment Type
alt	Alternative multiple fragments: only the one whose condition is true will execute.
opt	Optional: the fragment executes only if the supplied condition is true. Equivalent to an alt only with one trace.
par	Parallel: each fragment is run in parallel.
loop	Loop: the fragment may execute multiple times, and the guard indicates the basis of iteration.
region	Critical region: the fragment can have only one thread executing it at once.
neg	Negative: the fragment shows an invalid interaction.
ref	Reference: refers to an interaction defined on another diagram. The frame is drawn to cover the lifelines involved in the interaction. You can define parameters and a return value.

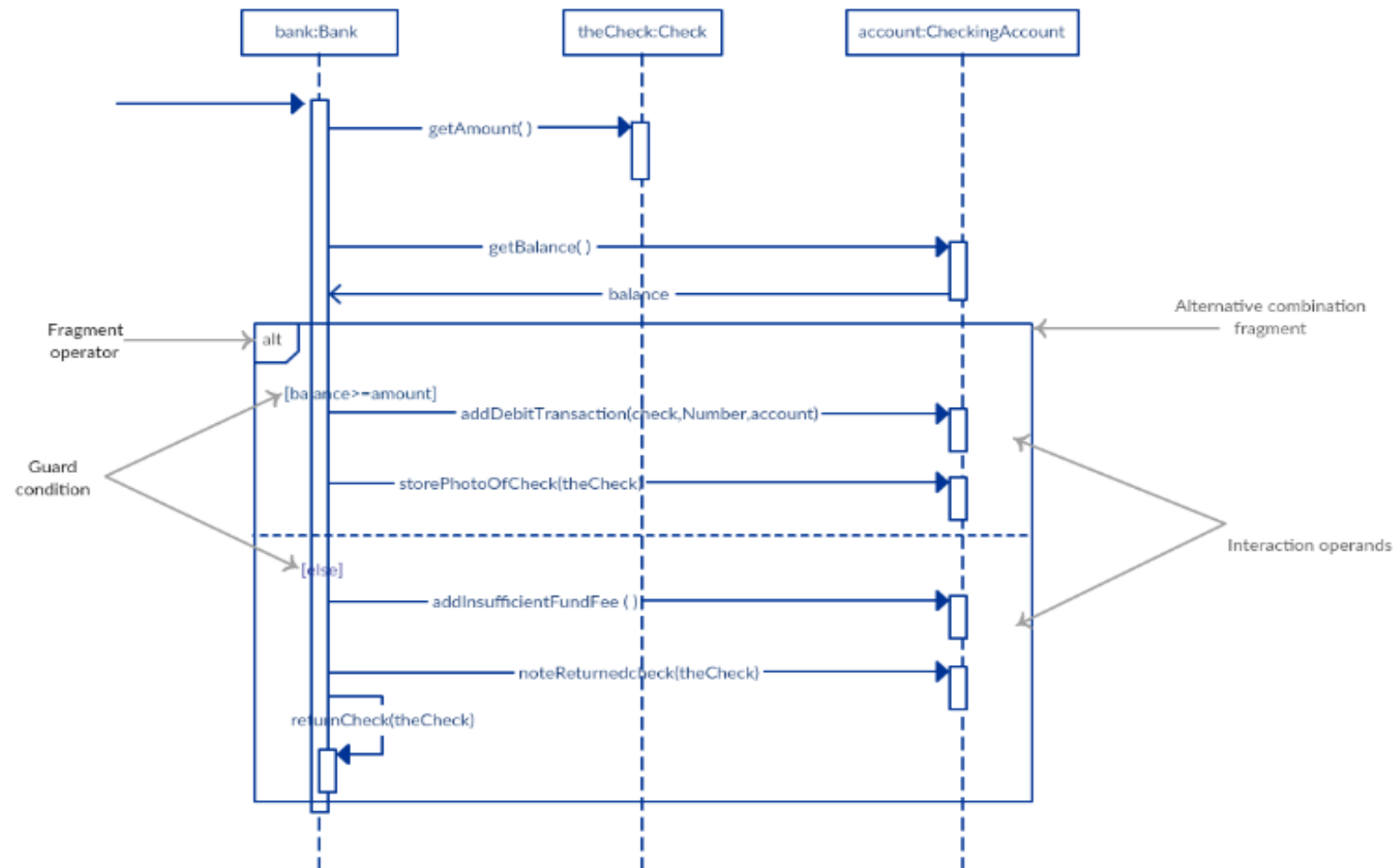
Sequence Fragments

- To represent alternatives, use the labeled rectangle shape with a dashed line inside.

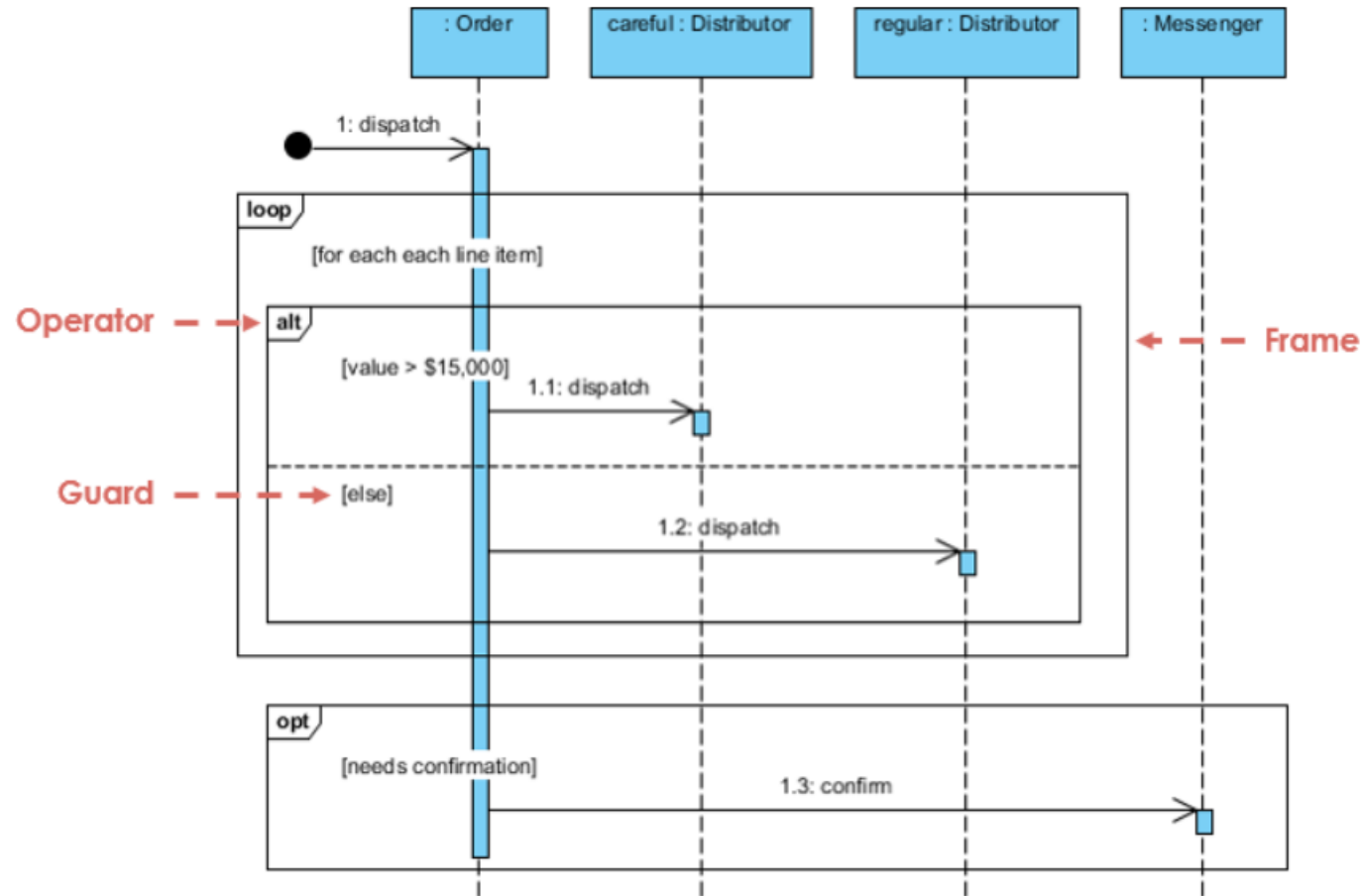


Alternative
symbol

Alternative Symbol Example

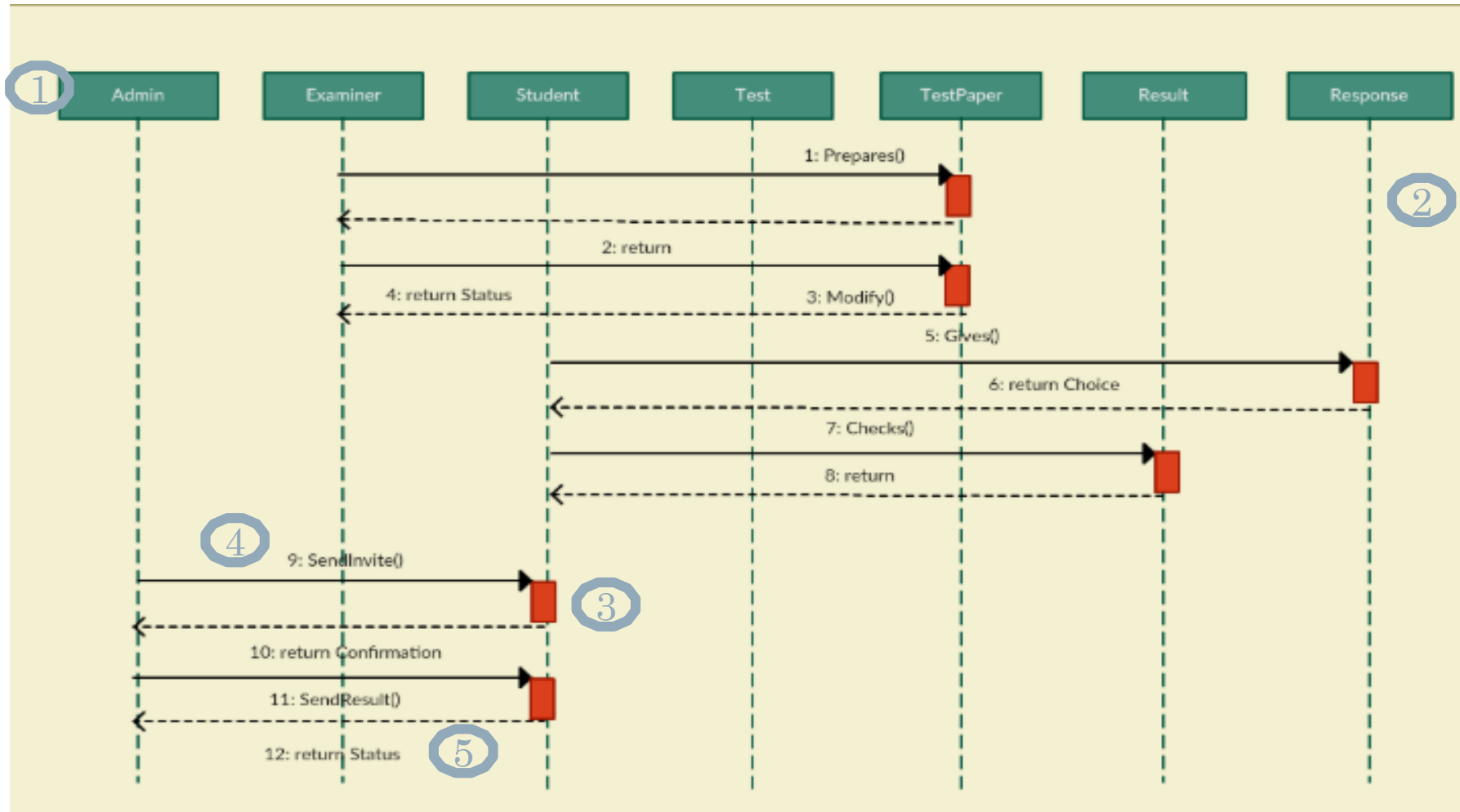


Combined Fragment Example



Activity

Identify the Elements



Consider the following use case narration describing the scenario of a specially designed telephone box used to take calls using a credit card.

Draw a sequence diagram to represent the scenario.

Use Case Name :	Take a telephone call using the credit card
Participant:	User Credit card processing center
Entry Conditions:	The user has a valid credit card The credit balance should be sufficient for the minimum amount required to take a phone call.
Exit Conditions:	User hangs the receiver Credit card is returned to the user
Flow of Events:	
1. The user inserts the credit card.	
2. Telephone box sends the message to the card center.	
3. Card center processes the card.	
4. Card center sends the available credit to the Telephone box.	
5. Telephone box processes the available balance.	
6. Telephone box shows a message for the user to pick up the receiver.	
7. User picks up the receiver.	
8. Telephone box displays the number pad to the user on the screen.	
9. User dials the number and makes the phone call.	
10. Telephone box processes the duration/amount.	
11. User ends the phone call by pressing the ‘END’ button / hanging the receiver.	
12. Telephone box displays the total phone call duration and the amount.	
13. Telephone box sends the amount to the card center.	
14. Card center charges the amount from the credit card.	
15. Card center sends the confirmation to the telephone box.	
16. Telephone box displays the message ‘THANK YOU, COME AGAIN’.	
17. Telephone box ejects the credit card.	
Exceptional conditions and alternative flow of events:	
When the credit is not sufficient to make a call: Display error message and Go to 17	
When the amount exceeds the credit card balance: Ends the phone call and Go to 12	